

:LiComputer: 1. Eliminación de dependencias LLM en el código

Para lograr un comportamiento **determinístico**, es necesario **remover todas las llamadas a modelos de lenguaje (LLM)** en el backend de MunBOT. En la rama `main` actual, el orquestador (`mcp-core`) invoca un microservicio `llm_docs-mcp` tanto para clasificar intenciones como para generar respuestas con RAG. Estos puntos deben ser modificados:

- **Orquestador (mcp-core):** En `orchestrator.py`, eliminar el uso de `LlmDocsClient` y sus métodos. Actualmente se llama a `llm_client.classify_intent()` para decidir la intención del usuario [GitHub](#), y luego, para preguntas informativas, se invoca la herramienta `doc-generar_respuesta_llm` del microservicio LLM [GitHub](#). **Solución:** Reemplazar esa lógica por una clasificación interna 100% **rule-based** (ver punto 2) y por consultas directas a la base de conocimiento. Concretamente:
 - Suprimir la función remota `classify_intent_remotely` y en su lugar usar la función `_heuristic_classify` local para todas las entradas (o invocarla directamente en `handle_turn`). Esto asegura que la intención se determine sólo con reglas predefinidas y no con el modelo [GitHub](#) [GitHub](#).
 - En la función `handle_document_query`, **eliminar** la llamada a `call_tool_microservice("doc-generar_respuesta_llm", ...)` [GitHub](#). En vez de enviar la pregunta al servicio LLM para obtener una respuesta, implementar una **búsqueda determinística** en la base de conocimientos (por ejemplo, recorrer documentos JSON en memoria para encontrar la respuesta apropiada; ver punto 3).
 - Deshabilitar las utilidades que usaban LLM para extracción de datos. Por ejemplo, la función `extract_name_with_llm` realizaba un prompt a un modelo para extraer el nombre del usuario [GitHub](#); se debe eliminar ese prompt y confiar únicamente en las regex ya definidas (la función ya contempla regex para nombres comunes [GitHub](#)). De igual forma, eliminar `remote_llm_generate` (si existiera) y cualquier llamado similar en `extract_email_with_llm` [GitHub](#) para no depender de generación de texto.
 - **Configuraciones:** Remover variables de entorno relacionadas con LLM en `.env` y `docker-compose.yml`. Por ejemplo, `LLM_DOCS_MCP_URL`, credenciales API y la dependencia del contenedor `llm_docs-mcp` pueden eliminarse del Compose. El servicio `llm_docs-mcp` y la base vectorial Qdrant ya no serán necesarios, por lo que pueden quitarse del despliegue para simplificar. En `docker-compose.yml` habría que borrar la sección del servicio `llm_docs-mcp` y la dependencia en `mcp-core` [GitHub](#) [GitHub](#). Con esto, también se puede reducir el tiempo de *startup* (antes se esperaba ~300s a que cargara el modelo LLM [GitHub](#)).
 - Asegurarse de que el **modo agente** o *canary* no invoque rutas generativas ocultas. Por ejemplo, la variable `AGENT_MODE` o `AGENT_CANARY_RATIO` debería configurarse para no usar ningún “modo GPT” alternativo. Dejar `AGENT_CANARY_RATIO=0` (por defecto) está bien. En resumen, toda ruta de código que implique generación abierta debe estar removida o inalcanzable.
- **Microservicio de agendamiento (scheduler-mcp):** Revisar que este servicio no dependa de LLM. En la implementación actual, *scheduler-mcp* sólo expone endpoints determinísticos: listar horas disponibles, reservar, confirmar, cancelar citas, etc. No se encuentran llamadas a ningún modelo lingüístico aquí – el microservicio espera parámetros estructurados (fecha, hora, nombres, etc.) y opera sobre la BD. Por lo tanto, **no requiere cambios sustanciales** aparte de asegurar que siempre reciba datos ya procesados. Antes, la interpretación del texto libre de fecha/hora la hacía el orquestador con `parse_date_time` (que usa reglas y utilidades de fechas), lo cual se mantiene. Sí conviene verificar que no queden restos de lógica para “entender” lenguaje natural: por ejemplo, si hubiera algún intento de interpretar texto de motivo de cita con IA (no parece haberlo). Dado que el orquestador ahora siempre le mandará datos limpios (fecha en AAAA-MM-DD, hora HH:MM, etc.), *scheduler-mcp* funcionará determinísticamente.
- **Microservicio de reclamos (complaints-mcp):** Este servicio ya funciona mayormente por reglas. Anteriormente podía haber dependido de LLM para analizar el texto del reclamo, pero en la versión actual se usa un clasificador simple de departamento por palabras clave [GitHub](#). **Acciones a tomar:**
 - Confirmar que no se usen LLM para nada. En `complaints-mcp/api.py` se observa que al registrar un reclamo, si no se indicó departamento, se llama a `clasificar_departamento(mensaje)` para inferirlo [GitHub](#). Esa función usa listas de palabras clave por categoría (Seguridad, Obras, Ambiente, Otros) [GitHub](#) – esto **ya es determinístico** y está bien. No se hace ningún llamado externo; por tanto, mantener este enfoque.
 - Asegurarse de que todos los campos requeridos se obtienen por flujo de conversación o se validan con reglas. El microservicio devuelve errores con `pending_field` si falta nombre, rut, email o mensaje [GitHub](#), y esos casos serán manejados por el orquestador con preguntas de seguimiento en vez de un LLM. No se necesita ninguna IA para validar formatos: hay regex para correo y RUT tanto en el microservicio [GitHub](#) como en el orquestador [GitHub](#).
 - Remover cualquier resto de integración con LLM. Por ejemplo, si existiera lógica para “resumir” el reclamo o extraer entidades usando un modelo, se debe quitar. En la lectura de código no se evidenció nada de eso, así que probablemente no haya cambios que hacer aquí.

En resumen, tras estos ajustes el backend **no llamará a ningún modelo generativo**. Todo procesamiento será con métodos nativos (regex, búsqueda en texto, condiciones) y llamadas directas a los microservicios de datos. Esto aumentará la predictibilidad: mismas entradas siempre producirán mismas salidas, sin depender de la aleatoriedad o tiempos de respuesta de GPT.

2. Enrutamiento basado en reglas

Con la eliminación de la clasificación por IA, el sistema debe **reconocer las intenciones del usuario mediante keywords, reglas fijas o regex**. La idea es replicar (y si es necesario ampliar) la lógica de `_heuristic_classify` que ya existía como respaldo. Actualmente, esa función realiza una serie de *if/elif* evaluando la frase normalizada del usuario [GitHub](#) [GitHub](#). Se debe estructurar así:

- **Priorizar intenciones explícitas (palabras clave únicas):** Por ejemplo, si el usuario menciona “*reclamo*” o “*denuncia*”, inmediatamente asumir intención **reclamo** (iniciar flujo de reclamo) [GitHub](#). Igualmente, términos como “*agendar*”, “*cita*”, “*reservar*” indican intención **agendar hora** [GitHub](#). Estas detecciones deben colocarse en la parte alta de la secuencia de reglas, para que no queden ensombrecidas por otras. En el código heurístico actual, efectivamente primero se chequea reclamo, luego agenda, etc., lo cual es correcto.
- **Reconocer saludos y conversación trivial (smalltalk):** Palabras como “hola”, “buenos días/tardes”, “adiós”, “gracias” deben mapearse a intenciones de **saludo, despedida, agradecimiento**, etc. Actualmente hay un set `SMALLTALK_PATTERNS` con muchas variantes comunes [GitHub](#) [GitHub](#), y la heurística marca intentos de saludo/despedida/agradecimiento usando esas palabras [GitHub](#). Conviene mantener y ampliar estas listas si hace falta, para cubrir expresiones coloquiales (p. ej. “hola bot”, “muchísimas gracias”, etc.). Esto garantiza que el bot responda adecuadamente con frases predefinidas de saludo o cortesía, sin confundirse con otras intenciones.
- **Intención “Agendar cita”:** Ya mencionado, detectar cualquier mención relacionada a reservas de hora: “*reservar hora*”, “*pedir turno*”, “*agendar cita*”, etc. [GitHub](#). Se puede usar un conjunto de sinónimos como el ya existente (agendar, cita, turno, hora, etc.). Esta regla disparará la intención `init_scheduler` para entrar al flujo de agendamiento. Es importante que esta detección ocurra **antes** de la detección de trámites/documentos, porque palabras como “hora” o “cita” podrían aparecer en preguntas informativas pero aquí indican acción.
- **Intención “Presentar reclamo”:** Similarmente, palabras como “*reclamo*”, “*queja*”, “*denuncia*” implican que el usuario quiere registrar un reclamo. Ya está contemplado con “reclamo”/“denuncia” [GitHub](#), pero podríamos añadir sinónimos como “queja” o frases como “reportar un problema”. Esta intención disparará `init_complaint` y el flujo de reclamos. Hay que colocar esta regla antes que la genérica de preguntas, y también antes que la de documentos/trámites. (En la heurística actual ya está antes que “tramite/documento”). Así, incluso si el usuario dice “*Quiero hacer un reclamo de bache en mi calle*”, la presencia de “reclamo” hará que se elija el flujo de reclamo, evitando confusión con la palabra “bache” que también podría relacionarse a info de obras. En caso de inputs ambiguos, definimos que **reclamo** tiene prioridad sobre trámite.
- **Intención “Consulta de información (FAQ/Trámite)”:** Si la frase no encaja en los casos anteriores pero menciona términos de trámites o documentos, se tratará como **pregunta informativa**. Por ejemplo, palabras como “*permiso*”, “*licencia*”, “*certificado*”, “*trámite*”, “*documento*” sugieren que el usuario pregunta por requisitos o detalles de un trámite [GitHub](#). La heurística actual devuelve `intent_documento` con `subintent_tramite` en estos casos. *En nuestro enrutamiento, podemos unificar estas bajo una etiqueta general tipo “consulta_tramite” o simplemente seguir usando documento/tramite. La idea es que el orquestador sepa que debe buscar en la base de conocimiento sobre trámites.* Además, si el usuario pregunta algo general como “¿Cómo hacer X trámite?” o “Información de tal permiso” que contiene palabras genéricas (“información”, “cómo”, “requisito”_ etc.), también se dirigirán por esta ruta. De hecho, la heurística ya contempla que si detecta palabras “información”, “municipalidad” o “pregunta”, y nada más específico, clasifica como **FAQ genérico** [GitHub](#). Esto cubre consultas tipo “Tengo una pregunta” o “Quiero información”. En nuestro diseño rule-based, podemos seguir este patrón: cualquier frase que no haya activado reclamo/agenda pero tenga pinta de pregunta genérica se clasifica como **faq**.
- **Respuestas de sí/no y otras confirmaciones:** El flujo actual maneja palabras de confirmación (`YES_WORDS` : “*sí, claro, por supuesto, dale*”) y negación (`NO_WORDS` : “*no, para nada, negativo, no gracias*”) [GitHub](#). Debemos preservar esto para que el bot entienda respuestas dentro de los diálogos (por ejemplo, cuando pregunta “¿Deseas registrarlo ahora?” el usuario puede responder “sí” o “no”). Estas listas de palabras ya están definidas; asegurarse de usarlas tal cual en la lógica (por ejemplo, marcando `intent_confirm` o `deny` según corresponda). También añadir variantes si hace falta (quizá “sip” como sí informal, etc., aunque no imprescindible para demo).
- **Estructura de las reglas:** Lo recomendable es implementar estas detecciones de manera **secuencial y excluyente**, tal como está en `_heuristic_classify`. Es decir: if (saludo) elif (despedida) elif (agradecimiento) elif (reclamo) elif (agenda) elif (tramite/doc) elif (faq genérico) ... etc, y si ninguna coincide, por defecto clasificar como FAQ genérico o `n/a`. De esta forma, la primera categoría que haga *match* con la frase del usuario determina la intención. La secuencia actual ya refleja una prioridad lógica (smalltalk primero, luego acciones, luego info) [GitHub](#) [GitHub](#). Podemos mantener ese orden.
- **Regex avanzados (opcional):** Para mayor robustez, podríamos usar regex para detectar frases completas típicas. Por ejemplo, una regex que capture “quiero (presentar|poner) un reclamo” o “quiero (agendar|reservar) una cita” podría marcar directamente esas intenciones. No obstante, dado el alcance del demo, quizás basta con la búsqueda de palabras clave. El riesgo de regex muy específicos es que no atrapen variaciones no contempladas; por eso, un buen conjunto de palabras clave puede ser suficiente. Se podría combinar ambos métodos: primero buscar algunas expresiones comunes con regex (p.ej. `r'^(quiero|deseo)\s+(hacer|presentar).*reclamo'` para reclamo) y si no, caer al chequeo de palabras. Esto no es estrictamente necesario si la lista de palabras clave es extensa.

En síntesis, el **router de intents** debe ser una serie de **reglas determinísticas** que analizan el mensaje del usuario y asignan una intención predefinida. Esto garantiza que *la misma frase siempre dispare el mismo flujo*. Es importante probar con variaciones de lenguaje natural para ver si alguna se cuela sin ser reconocida; de ser así, simplemente se agrega esa palabra/frase a la regla correspondiente. Por ejemplo, si un usuario dice “necesito agendar hora”, quizás la palabra “necesito” combinada con “hora” debe ser detectada; podemos incluir “necesito” en la lógica de agenda o al menos la palabra “hora” ya lo cubre. Esta afinación es manual pero dado que conocemos las 20-30 preguntas frecuentes esperadas (ver punto 5), podemos pre-cargar esas variantes. De hecho, muchas de esas variantes ya están listadas en los campos `user_says` y `alias` de la base de conocimientos [GitHub](#) [GitHub](#), lo cual nos sirve de guía de sinónimos a incluir en las reglas.

3. Adaptación de la base de conocimiento

Actualmente, la base de conocimiento de MunBOT está en varios archivos (JSON) dentro de `services/llm_docs-mcp/documents`, estructurada para RAG. Para el demo sin LLM conviene **reutilizar esa base pero accediendo de forma determinista**. Se recomiendan estos ajustes:

- **Unificación de la KB para acceso directo:** En lugar de consultar mediante embeddings y generación, cargaremos las preguntas y respuestas en memoria y haremos búsquedas exactas o por palabras clave. Por ejemplo, podemos **agrupar las FAQs y frases de ejemplo en un diccionario o estructura rápida** :

- Las *FAQs generales* (saludos, funcionamiento del bot, etc.) ya están en `RAG-faq.json`. Allí cada entrada tiene un campo `user_says` con variaciones de cómo el usuario podría formular la pregunta, y una `respuesta` fija [GitHub](#) [GitHub](#). Se puede procesar este JSON al iniciar el bot para crear un mapa de *expresiones clave* -> *respuesta*. Por ejemplo: mapear “¿Qué puedes hacer?” y sus variantes (“qué haces”, “qué sabes hacer”, etc.) a la respuesta “*Puedo orientarte en trámites, beneficios...*” [GitHub](#). Lo mismo con “¿En qué idiomas hablas?”, “¿Cuál es tu horario?”, “¿Cómo contacto a un humano?”, etc., que están en la lista [GitHub](#) [GitHub](#). De hecho, el JSON ya provee *alias* y *user_says* que podemos usar directamente para coincidir con la entrada correcta.
- Para las **FAQs de pequeños talk** (saludos, agradecimientos), ya vimos que hay respuestas definidas. Podríamos simplemente usar las respuestas hardcodeadas en código (como los arrays `GREETING_VARIANTS`, etc. que ya tenemos [GitHub](#) [GitHub](#)) o extraerlas del JSON también. Ambas opciones son válidas; mantenerlas en código podría ser más sencillo dado que ya están integradas en `_pick_smalltalk` [GitHub](#). En cualquier caso, las respuestas no requieren ningún LLM, solo seleccionar una variante (por simplicidad podemos usar siempre la primera variante o rotar si se quiere variedad).
- **Preguntas sobre trámites específicos:** La base de conocimiento está segmentada en documentos temáticos (trámites, departamentos, contribuciones, etc.). Para que el bot responda determinísticamente, podemos **predefinir las respuestas a preguntas típicas de esos temas**. Hay dos enfoques no excluyentes:

1. **Usar el contenido granular existente:** Por ejemplo, en `RAG-doc_tramites.json` están los fragmentos de información de cada trámite (requisitos, dónde se hace, horario, etc.) [GitHub](#) [GitHub](#). Sin un LLM, podríamos recuperar directamente esos textos. Por ejemplo, si el usuario pregunta “¿*Cuáles son los requisitos para el certificado de residencia definitiva?*”, nuestras reglas de intención la marcarán como `consulta_tramite` y extraerán el nombre del trámite (“certificado residencia definitiva”) y el aspecto preguntado (“requisitos”). Podríamos entonces buscar en `RAG-doc_tramites.json` la entrada cuyo “doc” es *certificado residencia definitiva* y cuyo tag/subcategoría es *requisitos*, y devolver el campo “texto” correspondiente (que en este caso enumera los requisitos específicos). **Implementación:** una función de búsqueda que recorra la lista JSON filtrando por doc y subcategoría. El orquestador ya tenía utilidades como `_extract_document_name` para aislar el nombre del trámite de la frase [GitHub](#) [GitHub](#), y `_needs_specific_info` para ver si falta especificar requisito/horario/etc [GitHub](#). Podemos seguir usando esa lógica:

- Si el usuario no especificó qué dato quiere (ej: pregunta solo “Información del trámite X”), el bot pedirá aclaración (esa parte ya estaba: “¿*qué información específica buscas, por ejemplo requisitos, horario, costo?*” [GitHub](#)).
 - Si especifica, construimos una clave (tramite + aspecto) para buscar.
 - En caso de encontrar el fragmento, devolvemos el texto. *Nota:* Estos textos son a veces muy concisos, ya que estaban pensados para ser integrados en una respuesta mayor generada por el LLM. Podríamos dejarlos tal cual o añadir un poco de contexto. Por ejemplo, en lugar de responder simplemente “*Lunes a viernes, 08:00–17:00.*” [GitHub](#), podríamos preprender “*El horario de atención es* ” para que suene más natural. Esto puede hacerse manualmente en ciertos casos. Sin embargo, dado el carácter demo, presentar la información puntual puede ser suficiente, asumiendo que el usuario entenderá que es la respuesta a su pregunta.
2. **Precompilar preguntas frecuentes explícitas sobre trámites/oficinas:** Otra forma más directa es anticipar qué consultas específicas va a hacer el usuario en el demo y añadirlas como entradas de FAQ explícitas. De hecho, en `RAG-faq.json` ya se incluyó al menos una: “¿*Qué requisitos necesito para la Cédula de Identidad?*” con su respuesta completa [GitHub](#) [GitHub](#). Vemos que enumeraron los requisitos en un solo texto más amigable (mencionando identificación, foto, pago, etc.). Esto demuestra que el equipo ya convirtió un caso de trámite en una pregunta-respuesta cerrada para no depender del armado dinámico por IA. Sería conveniente hacer lo mismo con unas cuantas consultas de trámites clave: por ejemplo, agregar FAQs tipo “¿Cuánto cuesta la licencia comercial?” o “¿Dónde tramito el permiso de piloto?” si esas fueran relevantes a Coruscant.
- Igualmente, para **direcciones de oficinas:** Podríamos incorporar preguntas estilo “¿*Dónde está el Departamento de Obras?*” y responder con la dirección exacta. El JSON `RAG-depto_info.json` seguro contiene direcciones de departamentos (por ejemplo “Edificio Consistorial nivel 1313” para Registro Civil). Podemos extraer de allí la información y crear respuestas completas. Si no se desea crear muchas entradas a mano, se puede hacer una búsqueda en ese JSON en tiempo real similar a los trámites: detectar el nombre del departamento y el hecho de que preguntó “dónde” o “dirección”, para devolver el campo de dirección. Pero tal vez más sencillo: definir unas pocas FAQs fijas para las dependencias más representativas:
Ej: “¿Cuál es la dirección del Departamento de Transporte Interestelar?” -> “Está ubicado en la Avenida Galáctica 123, nivel 5, Sector Federal.” (Ejemplo inventado). Dado que es un demo, podríamos cubrir 2–3 departamentos comunes.
 - **Horarios de atención:** Similar a direcciones. Muchas preguntas podrían ser “¿Cuál es el horario de atención de X oficina?” o “¿Hasta qué hora atiende la municipalidad?”. Esa información también está en la base (sea en `depto_info` o en un doc de horarios general). Conviene ofrecerla directamente. Podríamos incluir FAQs: “¿Cuál es el horario de atención de la Municipalidad?” – “Lunes a viernes de 8:30 a 17:00 hrs.”, etc. Y otra para “¿Atienden fines de semana?” – “No, el horario de atención es sólo en días hábiles.”, dependiendo de lo que se quiera mostrar. En `RAG-horario_comercio.json` quizás hay algo de horarios en situaciones especiales, pero para el demo mantenerlo simple.

- **Estructura de datos para búsqueda:** Una vez definidas las respuestas, hay que decidir cómo implementarlas. Podríamos:
 - Cargar los JSON en estructuras (listas/diccionarios) accesibles al orquestador. Por ejemplo, crear un diccionario `faq_answers` donde las claves sean posibles formulaciones (normalizadas) y el valor sea la respuesta. Para incluir sinónimos, podríamos mapear cada frase de `user_says` y `alias` a la misma respuesta. Así, si el usuario dice “como obtengo mi clave unica” o “cómo saco la Clave Única”, el bot buscará esa frase normalizada en `faq_answers` y encontrará la respuesta: “*Visita <https://claveunica.gob.cl...>*” [GitHub](#) (esa pregunta está en la base también). Este approach garantiza coincidencia exacta o por substring.

- Usar búsqueda *substrings* o *tokens*: Otra opción es iterar por las entradas de la base y ver cuál *encaja mejor* en la pregunta del usuario. Por ejemplo, podríamos determinar que si la pregunta del usuario contiene las palabras “requisitos” y el nombre de un trámite conocido, entonces devolver esa respuesta. Esto es similar a como haría un motor de búsqueda simple. Dado el reducido set de preguntas que manejaremos (20-30), se puede implementar con condicionales específicos o búsquedas lineales. Por ejemplo:

```
for faq in FAQs: if all(word in user_text for word in faq.trigger_words): return faq.answer
```

 donde `faq.trigger_words` es una lista de palabras clave importantes de la pregunta original. Esta técnica puede funcionar para preguntas tipo "¿Qué requisitos... [X]?" o "¿Dónde está ... [Y]?". Sin embargo, hay que tener cuidado de no activar respuestas incorrectas por coincidencias parciales. Por eso, preferimos usar las frases de ejemplo exactas cuando sea posible (que es lo que sugerimos con `faq_answers`).
- **Uso de alias:** Los JSON proveen campos `alias` que en muchos casos duplican `user_says`, pero a veces agregan formas alternas. Asegurarse de incluirlos en la lógica de coincidencia. Por ejemplo, en agradecimientos se ve alias “se agradece” aparte de “gracias” [GitHub](#) [GitHub](#). Estos alias deben mapear igual a la respuesta de agradecimiento breve.
- **Consistencia y limpieza:** Al no haber un LLM que formule la respuesta, todo lo que responda el bot vendrá directamente de estos textos preescritos. Por ello:
 - Revisar la **redacción** de las respuestas almacenadas. Si algunas eran fragmentos telegráficos (pensados para IA), tal vez enriquecerlos ligeramente. Por ejemplo, la respuesta almacenada para la cédula de identidad era bastante completa y bien redactada [GitHub](#), eso está perfecto. En cambio, la respuesta para “¿Cuál es tu horario de atención?” es “*Estoy disponible 24/7, siempre listo para ayudarte.*” [GitHub](#), lo cual funciona bien en tono conversacional. La mayoría de las respuestas FAQ parecen adecuadas, así que probablemente no requieren edición.
 - **Eliminar referencias a IA** en las respuestas si las hubiera. En un demo determinístico, el bot no debe mencionar nada de “como IA” o “modelo generativo”. Dado que la temática es municipal, no se ve nada de eso en las respuestas (quizás salvo “soy un Chatbot” en algún saludo, lo cual está bien).
 - **Suggested replies:** Algunas entradas tienen `suggested_replies` (p. ej. después de un agradecimiento breve se sugieren botones "Trámites", "Agendar cita", etc. [GitHub](#) [GitHub](#)). Esto sugiere que el front-end web muestra opciones rápidas. Asegurarse de seguir proporcionando esas sugerencias cuando corresponda. Sin LLM, tendremos que adjuntarlas manualmente con la respuesta. Por ejemplo, podemos conservar el campo `suggested_replies` de la FAQ y enviarlo junto con la respuesta JSON al front-end para que se muestren los botones. En WhatsApp estos sugeridos quizás no se muestren de igual forma (ver punto 4), pero en Web enriquecerán la demo.

En resumen, la base de conocimiento actual **ya cubre la mayoría de las preguntas frecuentes** necesarias; simplemente cambiaremos la forma de accederla: de un modelo generativo a una búsqueda determinista. Esto implica extraer las respuestas directamente de los JSON o de estructuras equivalentes. La ventaja es que el contenido puede seguir mantenido en formatos fáciles (JSON/YAML) que permiten actualizaciones rápidas sin tocar código, solo que ahora el “matching” será por reglas. Es aconsejable probar cada entrada para verificar que nuestras reglas efectivamente la encuentran. Por ejemplo, validar que si el usuario escribe “cual es tu correo” (todo minúsculas, sin tilde), nuestro buscador encuentre la FAQ de “¿Cómo puedo contactar a un humano?” y devuelva el correo de contacto [GitHub](#). Esto podría requerir normalizar texto (quitar tildes, pasar a minúsculas, etc.), cosa que ya se hace en el orquestador con `normalize_text`.

4. Ajustes para integración en WhatsApp y Web (sin ambigüedades) ➡📱💻

Lograr que el demo funcione de forma **clara y consistente en ambos canales** requiere atender algunos detalles de formato, flujos y fallbacks:

- **Mensajes claros y consistentes:** Sin un LLM, todas las respuestas son predeterminadas, lo cual en general aumenta la consistencia entre canales. Aun así, hay que cuidar el tono y contexto:
 - En WhatsApp, suele ser preferible que los mensajes no sean excesivamente largos y se presenten de forma conversacional. Las respuestas de nuestra KB ya son concisas en su mayoría, por lo que no deberían tener problema. Por ejemplo, respuestas como “Puedes escribirnos al correo contacto@coruscant.gov para comunicarte con un agente.” [GitHub](#) caben perfectamente en un solo mensaje de WhatsApp.
 - En la interfaz web, esas mismas respuestas se verán bien en bocadillos de chat. Si hubiera necesidad de formatear (negritas, listas), podemos seguir usando Markdown básico. El orquestador tiene la función `adapt_markdown_for_channel` que deja intacto el formato para “web” y “whatsapp” [GitHub](#), y solo elimina markdown para otros canales no interactivos. Esto quiere decir que podemos usar **negritas** o viñetas en las respuestas y se verán en web; en WhatsApp, el Markdown simple (asteriscos para negrita, etc.) suele también renderizarse (WhatsApp soporta *bold*, *italic*, pero habría que verificar si la API de Evolution las envía tal cual). En caso de duda, mantener formato plano para WA.
 - Evitar menciones al canal en la respuesta. Por ejemplo, no decir “haz click en el botón” si estamos en WhatsApp, o “escribe tal cosa” si en web se ofrece botón. Idealmente, diseñar las respuestas y sugerencias para que funcionen en ambos. Por ejemplo, en web podríamos tener un botón “Enviar correo ahora” (como sugiere `suggested_replies` para contacto humano [GitHub](#)), mientras en WhatsApp solo podemos enviar el texto del correo y el usuario tendrá que copiarlo manualmente. Esto está bien siempre que el bot lo enuncie claramente (“escribenos al correo X”).
- **Eliminación de ambigüedades en flujos:** Con LLM fuera, los flujos de conversación son totalmente guiados. Debemos asegurarnos de que **no haya mensajes contradictorios ni dobles interpretaciones**. En particular:
 - **Flujo de reclamo:** Antes, si el usuario decía “quiero hacer un reclamo”, teníamos dos posibles respuestas: la heurística iniciaría el flujo preguntando “¿Deseas registrarlo ahora?” [GitHub](#), pero también había una entrada en RAG-faq que respondía “Claro. Indícame el motivo de tu reclamo...” [GitHub](#). No queremos duplicar ni confundir. La recomendación es **unificar la ruta**: dado que implementaremos el flujo estructurado, conviene eliminar (o no usar) la respuesta estática que salta directamente a pedir el motivo. En su lugar, cuando detectemos la intención reclamo, seguiremos el flujo programado:
 1. Bot: “Puedo ayudarte a registrar tu reclamo. ¿Deseas registrarlo ahora?” (confirmación) [GitHub](#).

2. Usuario: “sí” -> bot pide nombre, etc. / “no” -> bot cancela.

Esta secuencia está codificada en `handle_complaint_flow` [GitHub](#) [GitHub](#) y funciona igual en Web y WhatsApp. **¿Por qué quitar la respuesta FAQ?** Porque si devolviéramos “Claro, indícame el motivo...” de golpe, el flujo de orquestador quedaría desincronizado – el bot saltaría a preguntar el motivo *sin* haber registrado nombre y RUT primero. Dado que queremos recoger datos de contacto antes del mensaje, es mejor usar el flujo con confirmación. Por tanto, hay que asegurarse de que la entrada FAQ “Quiero ingresar un reclamo” no se use. Podemos simplemente quitar esa entrada del JSON para no hacer matching, o detectar esa frase como intención reclamo prioritariamente y omitir la búsqueda en FAQs en ese caso. Lo importante es que el bot no mezcle dos enfoques distintos.

- **Flujo de agendar cita:** Algo similar ocurre con “Quiero agendar una cita”. Hay una FAQ que responde “Indícame el trámite y la fecha/hora deseadas...” [GitHub](#). Por otro lado, el flujo programado en `handle_scheduler_flow` asume que primero pedirá fecha, luego hora por separado [GitHub](#) [GitHub](#). Aquí debemos decidir una ruta consistente. Quizás podríamos **simplificar el flujo de agenda** para la demo: en lugar de dos preguntas (fecha y luego hora), podríamos aprovechar la respuesta estática para pedir ambos en un solo mensaje, como sugiere la FAQ. Por ejemplo, cuando detectemos intención *agenda*, el bot responda: “¡Claro! Por favor indícame el **trámite** para el cual quieres agendar y la **fecha y hora** que te acomoda.”. Esto tiene la ventaja de menos turnos; sin embargo, implica entender la respuesta del usuario que combine ambos datos. Dado que ya tenemos un parser de fecha/hora robusto, podríamos intentarlo: el usuario podría contestar “Necesito renovar licencia, el 5 de noviembre a las 10 AM”. El orquestador podría extraer fecha y hora de ahí (posiblemente ignorando la parte “renovar licencia” o usando eso para inferir departamento). Esto sería ideal pero la lógica actual no parsea el texto del trámite en la misma entrada. Alternativamente, mantener el flujo actual (preguntar fecha y luego hora por separado) es más guiado y seguro.
 - **Recomendación:** Mantener el flujo en dos pasos, pero **no usar** la respuesta estática duplicada. Es decir, cuando el usuario diga “quiero agendar cita”, el orquestador directamente pregunta “Para agendar una cita necesito que me indiques la fecha que te acomoda.” [GitHub](#). Esto ocurre hoy solo si el usuario ya disparó el intent agenda. Si también existe la FAQ “Quiero agendar una cita”, habría doble respuesta (lo cual generaría confusión). Por ello, igual que con reclamo, es mejor desactivar la respuesta FAQ para esa frase, y dejar que el flujo maneje la conversación. Así, en cualquier canal, el usuario recibirá una única respuesta coherente.
 - **Sin ambigüedad de contexto:** Un detalle a cuidar es que en WhatsApp el usuario podría escribir la fecha y hora en un solo mensaje (“el 10 de noviembre a las 9:00”). Nuestro flujo actual espera primero solo fecha, luego solo hora. Si recibe ambas a la vez en la respuesta a la primera pregunta, es posible que `parse_date_time` extraiga la fecha y hora de un tirón. De hecho, la función `parse_date_time` intenta devolver ambos si los encuentra en el texto. Si así sucede, el código posiblemente llenaría ambos en `state` y pasaría a consultar disponibilidad inmediatamente, saltándose la segunda pregunta. Esto no es necesariamente malo – podría verse como una interacción eficiente soportada “accidentalmente”. Habrá que probarlo. En todo caso, el sistema debería manejarlo sin romperse. (El peor caso es que el bot pregunte “¿a qué hora?” cuando en realidad ya entendió la hora; pero en la siguiente iteración igual avanzará).

En conclusión, para **evitar ambigüedades**, se debe **eliminar entradas de conocimiento que dupliquen las interacciones guiadas**. Priorizar los flujos programados (reclamo, agenda) sobre respuestas estáticas para esos intents. Así el comportamiento será consistente: el bot siempre seguirá el mismo libreto en esos casos en WA y Web.

- **Fallbacks unificados:** Cuando el bot no logre entender o encontrar respuesta, debe comportarse igual en ambos canales:
 - El mensaje de *fallback* genérico definido es: “Lo siento, no he entendido tu consulta. ¿Podrías reformularla?” [GitHub](#). Esto es neutral y funciona en cualquier medio. Asegurarse de que el orquestador devuelva exactamente ese texto (o la versión que queramos) cuando la intención sea desconocida (`intent == n/a`). Al quitar LLM, quizás pocas veces lleguemos a `n/a`, porque la heurística clasifica desconocidos como “faq”; sin embargo, puede ocurrir que no haya resultado en la base de conocimiento para una pregunta clasificada como FAQ. En ese caso, tras intentar la búsqueda determinista, debemos retornar un fallback. Hay que implementar esa comprobación: si ninguna regla de respuesta coincide, o si un *lookup* en la base de conocimiento no arrojó nada relevante, entonces enviar el mensaje de disculpa. Esto garantiza que el usuario no quede en silencio.
 - **Contador de fallback y escalamiento:** El código tiene lógica para, tras 3 fallbacks consecutivos, “escalar a humano” [GitHub](#). Esto lo conserva en `context_manager`: cuenta los fallos y en el tercero responde: “Un experto te contactará para continuar.” [GitHub](#). Podemos mantener esto, pues es útil para demo (muestra que tenemos una ruta de derivación). Pero debemos calibrar la expectativa: en un entorno real, en WhatsApp tal vez se derivaría a un operador humano; aquí solo diremos esa frase. Asegurarse de que el correo de contacto humano se proporcionó antes (como vimos en las FAQs, el usuario puede preguntar manualmente).
 - Es importante que el fallback no se active erróneamente cuando sí hay respuesta. Con la base de 20-30 FAQs, deberíamos cubrir la mayoría de consultas esperables. Realizar pruebas de “stress” en ambos canales: por ejemplo, enviar en WhatsApp una pregunta que no esté en la base y ver que el bot responde con la frase de fallback apropiada. Luego enviar otra pregunta válida y confirmar que el conteo se resetea (el código actual lo hace si el usuario dice que la respuesta fue útil, pero podríamos simplificar asumiendo que una nueva pregunta exitosa borra el contexto de fallos).
- **Formato de sugerencias y opciones:** En la web, seguramente el frontend de chat interpretará campos como `suggested_replies` y los mostrará como botones. En WhatsApp, esos sugeridos no existen visualmente, pero podríamos aún listarlos manualmente en el mensaje (“Opciones: 1. Trámites, 2. Agendar cita, ...”). Sin embargo, dado que es un demo, probablemente no necesitemos listar opciones numéricas – es más sencillo que el usuario escriba libremente. Es suficiente con que en Web estén los botones para guiar. Asegurarse de que la presencia de esos botones no confunda la conversación: por ejemplo, tras un agradecimiento, el bot ofrece sugerencias de próximos temas; en WhatsApp el usuario no los ve, pero podría igual preguntar otra cosa. No hay mayor problema allí.
- **Pruebas en ambos canales:** Es esencial probar todos los flujos tanto en WhatsApp (vía Evolution API) como en la Web. La experiencia debe ser similar:
 - Verificar que los **mensajes multilínea** se muestren correctamente. Por ej., si alguna respuesta contiene saltos de línea (como la de departamentos que enumeraba opciones numeradas en la pregunta de reclamo: “1. Alcaldía, 2. Social, ...” [GitHub](#)), en Web

eso aparece con formateo de nueva línea; en WhatsApp, también debería pero hay que ver si hay que ajustar. Quizá usar `\n` explícitos está bien (en el JSON ya vienen).

- **Emojis y tildes:** Asegurar que caracteres especiales se ven bien. El bot a veces usa símbolos de exclamación invertida (¡, ¿). En Web sin problema; en WhatsApp, también se ven. Ningún ajuste necesario, solo no quitarlos.
- **Sesiones y contexto:** Confirmar que el contexto de la conversación se mantiene separado por usuario. Evolution API generalmente asociará cada número de WhatsApp a un `session_id`. El orquestador ya maneja `session_id` para contexto. Debemos probar que, por ejemplo, si un usuario WhatsApp inicia un reclamo y no responde, y luego envía otro mensaje no relacionado, el bot quizás siga esperando el nombre (porque `current_flow` sigue activo). Si ese usuario dijo “cambiamos de tema”, el bot lo detectaría y limpiaría el contexto [GitHub](#). Es bueno mencionar al usuario esa opción en caso de confusión: de hecho, “*cambia de tema*” está en patrones de `CHANGE_TOPIC_PATTERNS` [GitHub](#). En Web, como la sesión podría reiniciarse al recargar, es menos problemático. En WhatsApp, la sesión es persistente, así que debemos **manejar la cancelación**: ya se hace en reclamos con la palabra “cancelar” [GitHub](#). Para agenda, no se implementó un “cancelar”, pero dado que es corto, tal vez innecesario. Aún así, podríamos añadir “cancelar” para salir de flujo de agenda también, de forma simétrica.
- **Fallback de canal:** Si algo llegara a fallar técnicamente en uno de los microservicios (por ejemplo, falla la conexión a scheduler), el orquestador devuelve un mensaje de error genérico: “*No pude consultar la agenda en este momento...*” [GitHub](#). Es bueno verificar que esos mensajes también sean entendibles. Similar con reclamo: el microservicio retorna un mensaje de éxito con un comprobante (texto de recibo) [GitHub](#). Revisar que en WhatsApp ese comprobante (que puede incluir saltos de línea con detalles) se vea bien formateado. Si no, quizás formatearlo con line breaks explícitos `\n` (ya lo hace). Esto entra en pruebas integrales.

En conclusión, la integración requiere principalmente **consistencia en el diálogo**. Eliminando respuestas duplicadas y manteniendo un solo flujo por intención, nos aseguramos que el bot no dé mensajes conflictivos. Además, los *fallbacks* y mensajes de sistema deben ser los mismos para WhatsApp y Web. Con un set fijo de respuestas, podemos previsualizarlas en ambos canales y ajustarlas manualmente si alguna se ve mal (cortes de línea, longitud). La ausencia de LLM evita variaciones sorpresivas, así que la experiencia será más predecible – lo cual es lo que se busca en este demo determinístico.

5. Cobertura mínima de 20–30 FAQs 🎯

Para el demo “Gobierno de Coruscant” es fundamental cubrir al menos **20 a 30 preguntas frecuentes** con respuestas útiles. Afortunadamente, gran parte de esas FAQs ya están identificadas en la base de conocimiento. Para asegurarnos de la cobertura, debemos:

- **Listar las FAQs esenciales y verificar sus respuestas:** Basado en la descripción del proyecto, las preguntas deben abarcar:
 - **Requisitos de trámites:** Ejemplos:
 - “¿Qué requisitos necesito para [tramite X]?” – Como el de la Cédula de Identidad [GitHub](#). Podríamos cubrir también “permiso de aterrizaje”, “licencia de piloto” u otros trámites ficticios mencionados en la documentación [GitHub](#). Por ejemplo, si existe “permiso de aterrizaje interplanetario” en Coruscant, incluir una FAQ con sus requisitos. Cada trámite importante idealmente tendría un Q&A: requisitos, costo, vigencia, etc., pero para demo tal vez con uno o dos ejemplos concretos basta. En la base actual, además de cédula, no vi otra pregunta explícita de requisitos – podemos añadir una.
 - “¿Dónde obtengo [tal certificado/licencia]?” – Preguntas de ubicación de trámite. Ejs: “¿Dónde puedo sacar la licencia de piloto?” con respuesta “En el Departamento de Transporte Interestelar, ubicado en...”. Esta información está en los documentos de trámites (`donde_obtener`). Se puede contestar directamente. Tener al menos un ejemplo.
 - “¿Cuánto cuesta [tal trámite]?” – Si es relevante. Podría ser que en Coruscant algunos trámites tengan tasa. Si hay datos en RAG (p. ej. en Contribuciones), se puede armar una FAQ: “¿Cuánto cuesta la patente comercial?” -> “La tasa municipal anual es XX créditos.”.
 - “¿Cuál es la vigencia de [licencia X]?” – Ejemplo: Licencia de piloto dura N años. En `RAG-doc_tramites` hay fragmentos de *vigencia* para ciertos docs [GitHub](#). Podríamos transformarlo en pregunta: “¿Por cuánto tiempo es válido el certificado de residencia?” – Respuesta: “Tiene vigencia de 1 año” (como dice el texto [GitHub](#)).
 - **Direcciones de oficinas:** Ejs:
 - “¿Dónde queda la oficina de [Departamento Municipal X]?” – Responder con la dirección. Importante incluir la Alcaldía o edificio principal. Por ejemplo: “¿Dónde está la Municipalidad de Coruscant?” – Respuesta: “En Coruscant Central, Plaza Imperial, Nivel 500.” (inventado).
 - “¿Cómo contacto al Departamento de Medio Ambiente?” – Podría ser otra: respuesta con quizás un correo o teléfono, si existiera. En `depto_info` se mencionan contacto/correo [GitHub](#). Para demo, tal vez no entremos en teléfonos, pero un correo general podría darse.
 - **Horarios de atención:** Ejs:
 - “¿Cuál es el horario de atención de la Municipalidad?” – Como se dijo, lunes a viernes 8:30-17:00 (definirlo y mantenerlo consistente en respuestas).
 - “¿Atienden los sábados?” – Respuesta: “No, el gobierno municipal atiende solo de lunes a viernes.” (Si es el caso).
 - Quizá “¿Horario de [departamento específico]?” – Posiblemente todos tengan el mismo horario, se puede responder genérico.
 - En `RAG-horario_comercio` se mencionaban horarios de comercio y ferias [GitHub](#), pero esas pueden ser extra. Si se quiere, una pregunta podría ser “¿Hasta qué hora pueden abrir los comercios en Coruscant?” – respuesta basada en ese reglamento (pero es más específica, tal vez fuera del demo de atención ciudadana).
 - **Preguntas sobre el chatbot mismo (FAQ de funcionamiento):**
 - “¿Qué puedes hacer?” – Está cubierta [GitHub](#).
 - “¿Qué tipo de consultas puedo hacer?” – También en la base [GitHub](#).

- “¿En qué idiomas hablas?” – En la base [GitHub](#) (responde que sólo español).
- “¿Cuánto cuesta MunBOT?” – Incluida [GitHub](#), esta es meta (sobre el producto).
- “¿Cómo funciona este chatbot?” – Podría estar implícita en “qué puedes hacer”; si no, podríamos añadir algo sobre que funciona por reglas en este demo (aunque al usuario final quizá no le importa esa precisión). Probablemente no sea necesario entrar en detalles técnicos al usuario, así que omitible.
- “¿Cómo hablo con un humano?” – Incluida [GitHub](#). Importante porque demuestra la opción de escalar.
- “Tengo otra consulta” – Incluida [GitHub](#) como forma de continuar la conversación educadamente.
- Saludos y despedidas varias – ya cubiertos ampliamente.
- Agradecimientos – cubiertos [GitHub](#).
- Estado de ánimo del usuario – hay respuestas para cuando el usuario dice “estoy bien” o “estoy mal” [GitHub](#) [GitHub](#). Eso es un plus interesante del bot. Se mantendrán.

- **Verificar que alcanzamos 20-30:** Contando los tipos anteriores, fácilmente llegamos a ~25. De hecho, el JSON `RAG-faq.json` ya tiene ~26 entradas [GitHub](#) [GitHub](#), sin contar las potenciales adicionales de trámites específicos que agreguemos. Es recomendable repasar esas entradas una por una y marcarlas como *incluidas*:

1. Saludos (básico, compuesto) – 2
2. Despedidas (corta, con cierre) – 2
3. Agradecimientos (breve, por la ayuda) – 2
4. Respuestas a “¿Cómo estás?” (estado de bot) – 1
5. Reacción a “estoy bien/mal” (estado usuario positivo/negativo) – 2
6. “¿Qué puedes hacer?” – 1
7. “¿Cuál es tu horario de atención?” – 1
8. “¿En qué idiomas hablas?” – 1
9. “¿Cómo protegen mis datos?” – 1 (hay una entrada sobre datos personales [GitHub](#)).
10. “¿Cuánto cuesta MunBOT?” – 1
11. “Quiero ingresar un reclamo” – (la entrada existe pero decidimos no usarla; en su lugar el flujo se hace cargo, pero contaría como caso de uso cubierto: registrar reclamo)
12. “Quiero agendar una cita” – (igual que reclamo, cubierto vía flujo en vez de FAQ directa)
13. “¿Cómo obtengo la Clave Única?” – 1 (incluida en base [GitHub](#)).
14. “¿Qué tipo de consultas puedo hacer?” – 1
15. “Quiero obtener un certificado/licencia/permiso/patente” – 4 (estas entradas en la base devuelven preguntas de contra: “¿Qué [documento] en particular?” [GitHub](#) [GitHub](#), es decir, son para manejar cuando el usuario pide algo genérico. Son útiles para no fallar ante “quiero un permiso” – el bot pedirá aclaración. Mantenemos estas reglas para que el bot no se quede callado; equivalen a mini-fallbacks contextuales).
16. “¿Qué requisitos necesito para la Cédula de Identidad?” – 1 (esta es un ejemplo concreto de trámite) [GitHub](#).
17. “¿Cómo puedo contactar a un humano?” – 1 [GitHub](#).
18. “Tengo otra consulta” – 1 [GitHub](#).

Eso suma alrededor de 25 entradas útiles. Si agregamos tal vez 2-3 sobre direcciones de departamentos y 1-2 sobre costos de trámites, estaremos alrededor de 30. Podemos, por ejemplo, añadir:

- “¿Dónde queda el Departamento de Registro Civil?” – Respuesta: “En el Edificio Consistorial, nivel 1313, Sector Federal.” (tomando la info de `RAG-doc_tramites` que lo menciona [GitHub](#)).
- “¿Cuál es el horario del Departamento de Transporte?” – Respuesta: “Lunes a viernes, 08:00–17:00 hrs.” (asumiendo todos igual).
- “¿Cuánto cuesta registrar un reclamo?” – Probablemente gratuito; se puede responder “Presentar un reclamo no tiene costo.” (Aunque esto tal vez no lo pregunten, lo menciono como idea de aprovechar info).
- “¿Qué pasa si no pago la tasa de aseo?” – Si quisiéramos usar `RAG-Contrib_Derechos.json` (penalidades), pero quizás eso es muy específico.

Enfocándonos en las más relevantes, la lista de ~25-30 queda cubierta.

- **Asegurar implementación:** Una vez identificadas, hay que **probar cada una** contra el bot:
 - Enviar exactamente la pregunta o variaciones a través del front de WhatsApp y web, y comprobar que obtenemos la respuesta esperada (ni fallback ni algo incorrecto).
 - Si alguna no funciona, ajustar la regla o añadir la clave que falta. Por ejemplo, si el usuario pregunta “donde puedo obtener licencia piloto” y no coincide porque nuestra entrada espera “obtener una licencia”, habrá que añadir sinónimos (“sacar”, “tramitar”) a la detección. Es un proceso iterativo pero manejable con 30 preguntas.
 - No olvidar los casos de conversación: saludar primero debe disparar saludo [GitHub](#), luego quizás usuario dice “quiero saber algo” – el bot preguntará qué; todo eso debe fluir bien.
 - Cobertura de negativos: si el usuario dice “no gracias” tras recibir ayuda, el bot debe despedirlo educadamente (eso está en despedida con cierre [GitHub](#)).
 - También testear combinaciones: e.g. “Hola, quiero saber horarios” en un mismo mensaje. Sin LLM, nuestro sistema ve “hola” y “horarios”. La heurística `_is_multi_intent` intentaba detectar múltiples intenciones en una frase [GitHub](#) [GitHub](#), pero en nuestro enfoque actual, probablemente tomará la primera coincidencia fuerte. En este ejemplo, “hola” activaría saludo y respondería un saludo, posiblemente ignorando la parte de horarios. Esto es una limitación del determinismo: el bot puede responder solo a una cosa a la vez. **Recomendación:** En el demo, guiar a los usuarios a hacer preguntas una por una. No es crítico solucionarlo ahora, pero sí tener en cuenta. Podríamos documentar que el bot responde a una consulta a la vez.

En conclusión, **20–30 FAQs quedan cubiertas** al compaginar las ya existentes con unas pocas adicionales. Es crucial revisar que no falte ninguna categoría importante mencionada en el objetivo del proyecto. Tras implementar los cambios, MunBOT demo responderá correctamente a saludos, despedidas, agradecimientos, ~5–10 preguntas sobre su propio funcionamiento, ~5 de trámites (requisitos, dónde, horarios, costos si aplica), ~5 de departamentos (dirección, contacto, horarios), ~2 de procedimientos especiales (ClaveÚnica, etc.), y manejará el registro de **reclamos** y la **agenda de citas** de forma guiada y determinística. Todo esto sin intervención de modelos generativos, garantizando respuestas estables y predecibles en cada interacción.